TOXIC INGREDIENTS INDICATOR

An AI-Powered Cosmetic Safety Analysis Platform

Applied Research Project Report

Master of Science in Business Analytics
California State University, East Bay

Submitted by:

Samadrita Roy Chowdhury

nj7161

Spring 2026

Table of Contents

1. Introduction and Business Problem

1.1 Background and Motivation
1.2 Problem Statement
1.3 Objectives
1.4 Scope and Deliverables

2. Related Work

2.1 Consumer-Facing Cosmetic Safety Tools
2.2 Regulatory Frameworks and Reference Databases
2.3 Optical Character Recognition in Label Analysis
2.4 Large Language Models in Consumer Health Communication
2.5 Full-Text Search for Ingredient Matching
2.6 Gap Analysis and Positioning

3. Data Description

3.1 Data Sources and Collection
3.2 Database Schema
3.3 Ingredient Coverage and Chemical Family Classification
3.4 Hazard Severity Tiering
3.5 Data Quality and Curation

4. Models

4.1 System Architecture Overview
4.2 Technology Stack
4.3 ETL Model
4.4 Search and Retrieval Model
4.5 OCR Pipeline Model
4.6 Generative AI Model

5. Results

5.1 System Functionality Results
5.2 OCR Performance Results
5.3 Search and Retrieval Performance
5.4 Generative AI Output Quality
5.5 User Interface and Experience
5.6 Performance Benchmarks and Known Limitations

6. Conclusions and Impact

6.1 Summary of Findings

6.2 Business and Consumer Impact
6.3 Analytical and Technical Contributions
6.4 Reflections and Lessons Learned
6.5 Conclusion

7. Contributions

7.1 Project Author Contributions
7.2 AI Tool Contributions and Boundaries
7.3 Open-Source Dependencies

8. References

9. Product Lifecycle

9.1 Lifecycle Overview
9.2 Lifecycle Decisions and Rationale

10. Prioritization Framework

10.1 ESVF Methodology
10.2 ESVF Scoring Results
10.3 Prioritization Outcomes

11. Product Roadmap

11.1 Three-Horizon Framework
11.2 Plan of Record (0-6 Months)
11.3 Plan of Intent (6-12 Months)
11.4 Plan of Direction (12+ Months)
11.5 Roadmap Governance

12. Appendix - Source Code

A.1 Database - FTS5 Full-Text Search Index Initialization
A.2 Database - Two-Stage Ingredient Search
A.3 Search - Text Parsing and Domain Expansion
A.4 OCR - Dual-Path Image Recognition Pipeline
A.5 OCR - Postprocessing and INCI Error Correction
A.6 Generative AI - Core API Call Wrapper
A.7 Generative AI - Product Summary with Retry Logic
A.8 Generative AI - Per-Ingredient Explanation with Caching
A.9 Database - Hazard Record Retrieval

1. Introduction and Business Problem

1.1 Background and Motivation

The global cosmetics and personal care market surpassed $600 billion in 2023 and continues to grow at a steady pace, driven by rising consumer awareness of skincare, wellness, and beauty. Yet beneath the glossy packaging of these everyday products lies a persistent and under addressed problem: the widespread use of chemical ingredients that carry documented health risks. Parabens, formaldehyde-releasing preservatives, PFAS compounds, benzophenones, and phthalates are routinely found in products that consumers apply to their skin daily, often without any meaningful awareness of the associated risks.

The regulatory landscape does little to close this gap. The United States Food and Drug Administration does not require pre-market approval for most cosmetic ingredients. The European Union and California have taken more decisive action, with the EU banning over 1,300 substances and California's AB 2762 (Toxic-Free Cosmetics Act) prohibiting 24 high-risk chemicals. Even so, ingredient lists on product labels remain long, dense, and written in International Nomenclature of Cosmetic Ingredients (INCI) Latin terminology that the average consumer cannot interpret. The knowledge barrier between a product label and an informed purchasing decision is enormous.

This project addresses that gap directly. The Toxic Ingredients Indicator is a data-driven, AI-augmented web application that allows any consumer to scan or type a cosmetic ingredient list and instantly receive a structured, hazard-tiered analysis of every flagged chemical. By combining a curated toxicological database, optical character recognition, full-text search, and large language model explanations, the system transforms an otherwise impenetrable label into a comprehensible safety report.

1.2 Problem Statement

Consumers lack accessible, real-time tools to evaluate the safety of cosmetic ingredients at the point of decision. Existing resources such as the Environmental Working Group's SkinDeep database require users to search one ingredient at a time and navigate complex scientific scoring systems. No freely available tool exists that can accept a raw, unprocessed label, whether typed, pasted, or photographed, and return a complete, prioritized risk profile for the entire product in seconds.

The core business problem is one of information asymmetry. Manufacturers hold complete formulation knowledge. Regulators hold curated risk data. Consumers hold neither. The Toxic Ingredients Indicator acts as the bridge, translating both sources into plain, actionable language for the end user.

1.3 Objectives

This project was designed to achieve four primary objectives:

1. Build a comprehensive, structured database of cosmetic ingredients with associated hazard data, regulatory citations, severity tiers, and chemical family classifications.
2. Implement a multi-modal input pipeline that supports direct text search, paste-from-label, and image upload with OCR processing to accommodate real-world consumer behavior.
3. Integrate generative AI to produce plain-English product summaries and per-ingredient explanations, making technical toxicological data accessible to non-expert users.
4. Design the system with a product management framework that defines a clear product lifecycle, a prioritization methodology, and a forward-looking feature roadmap.

1.4 Scope and Deliverables

The scope of this project encompasses database design and population, backend search logic, image preprocessing and OCR pipeline development, generative AI integration, and a fully functional Streamlit web application. The database covers 736 flagged cosmetic ingredients organized across 30 chemical families and 5 severity tiers, with regulatory references drawn from the European Commission's cosmetic ingredient regulation, the FDA, California Proposition 65, and the IARC monograph series. The application was designed as a consumer-facing tool and does not require any scientific background to operate.

This report documents the full research process, technical implementation, product management decisions, and findings of the project. It also presents a forward-looking roadmap and architecture specification that positions the Toxic Ingredients Indicator as a scalable, extensible platform for cosmetic safety analysis.

2. Related Work

2.1 Consumer-Facing Cosmetic Safety Tools

Several tools exist that attempt to address consumer awareness of cosmetic ingredient safety, though each carries meaningful limitations that leave a significant usability gap in the market.

The Environmental Working Group's Skin Deep database is the most widely referenced resource in this space. It assigns a hazard score from 0 to 10 for thousands of cosmetic products and individual ingredients, drawing from scientific literature and regulatory lists. However, Skin Deep requires users to search one ingredient at a time or look up a specific product by brand name. It provides no mechanism for a user to input a raw, unparsed label from a product they have in hand, and its scoring methodology, while thorough, is not designed for consumer comprehension without background knowledge.

Think Dirty is a mobile application that allows users to scan a product barcode and retrieve a toxicity score. Its utility is limited to products that exist in its proprietary database, which means unregistered, generic, or international products return no results. It also does not support ingredient-level input or image-based label reading.

CosDNA and INCI Decoder both allow users to paste a full ingredient list and return per-ingredient annotations. These tools are closer in spirit to the Toxic Ingredients Indicator but lack OCR support, offer no AI-generated plain-English explanations, and present data in formats that still require scientific literacy to interpret. Neither tool performs hazard severity tiering or chemical family clustering.

The table below summarizes how the Toxic Ingredients Indicator compares to existing tools across key functional dimensions.

Tool	Input Method	OCR Support	AI Explanations	Batch Analysis
EWG Skin Deep	Manual search, one ingredient at a time	No	No	No
Think Dirty	Barcode scan or product name	No	No	No
CosDNA	Paste ingredient list	No	No	Partial
INCI Decoder	Paste ingredient list	No	No	Yes

Tool	Input Method	OCR Support	AI Explanations	Batch Analysis
Toxic Ingredients Indicator (This Work)	Search, paste, image upload, PDF	Yes (Tesseract + Gemini Vision)	Yes (Gemini 2.5 Flash)	Yes (up to 30 ingredients)

Table 1. Feature comparison of cosmetic safety analysis tools.

2.2 Regulatory Frameworks and Reference Databases

The toxicological foundation of this project is built on four primary regulatory and scientific sources, each representing a distinct jurisdiction and scope of authority.

The European Commission's Cosmetics Regulation (EC No 1223/2009) maintains one of the most comprehensive lists of restricted and prohibited cosmetic substances in the world, with over 1,300 banned ingredients and several hundred subject to use restrictions. This regulation forms the backbone of the hazard classification used in the Toxic Ingredients Indicator database, providing both the prohibited status and the scientific rationale for many high-severity entries.

California's Toxic-Free Cosmetics Act (AB 2762), signed into law in 2020, represents the most stringent cosmetic safety legislation in the United States. It prohibits 24 chemicals including mercury, formaldehyde, lead, and several PFAS compounds from cosmetics sold in California. Because California's market size effectively sets a de facto national standard for compliant formulation, this regulation is highly relevant to any US-facing safety analysis tool.

The International Agency for Research on Cancer (IARC) monograph series provides independent carcinogenicity classifications for chemical substances based on accumulated epidemiological and experimental evidence. IARC classifications (Group 1 through Group 3) are used throughout the hazard database to anchor severity ratings for ingredients with carcinogenic potential, including formaldehyde, certain hair dye precursors, and coal tar derivatives.

The FDA's cosmetic ingredient safety resources, including guidance on prohibited and restricted ingredients, provide the US regulatory baseline. The Modernization of Cosmetics Regulation Act of 2022 (MoCRA) has begun to expand federal oversight, making FDA citations increasingly relevant as a forward-looking reference source for this system.

2.3 Optical Character Recognition in Label Analysis

Optical character recognition is the process of converting images of text into machine-readable strings. Its application to product label analysis introduces several challenges that distinguish it from standard document OCR: curved surfaces, reflective packaging, small font sizes, densely

packed text, variable lighting in consumer photography, and the use of INCI nomenclature that standard OCR language models are not trained to recognize.

Tesseract OCR, originally developed by Hewlett-Packard and maintained by Google since 2006, is the most widely used open-source OCR engine. Tesseract 4 introduced an LSTM-based neural network recognition engine that significantly improved accuracy on printed text. The Toxic Ingredients Indicator implements a dual-path OCR pipeline using Tesseract as its primary engine, applying image quality detection to route inputs between a lightweight fast path and a heavier multi-variant preprocessing pipeline using CLAHE, Gaussian unsharp masking, deskewing, gamma correction, and contrast stretching.

Gemini Vision serves as a complementary OCR engine for label images that Tesseract struggles to parse. Its use follows emerging best practices in hybrid OCR systems, where outputs from a rule-based engine and a vision-language model are compared by word count and quality heuristics, with the higher-quality result selected automatically. This dual-engine approach substantially improves recall on dark, curved, or compressed label photographs without degrading performance on clean inputs.

2.4 Large Language Models in Consumer Health Communication

The application of large language models to health and safety communication has grown substantially since the public release of GPT-3 and the subsequent proliferation of accessible API-based generative AI services. Research in this area has found that LLMs can reliably translate clinical or scientific language into accessible consumer-facing text when provided with structured context and constrained output instructions.

Within the cosmetics and personal care domain, LLM applications remain relatively underexplored in published literature. The Toxic Ingredients Indicator contributes to this space by demonstrating a structured, constrained prompting strategy for ingredient explanation. Rather than asking the model to generate free-form safety advice, it supplies a structured context block containing the ingredient name, risk tier, chemical family, and a one-line hazard description, then instructs the model to produce exactly two plain-English sentences. This approach reduces hallucination risk and produces consistently structured, verifiable output aligned with the underlying database content.

The selection of Gemini 2.5 Flash reflects a deliberate design choice to prioritize low latency and cost efficiency. For summarization and explanation tasks of this scope, a mid-tier model with well-crafted prompts produces output quality comparable to larger models at significantly lower inference cost, consistent with recent literature on prompt engineering versus model scaling.

2.5 Full-Text Search for Ingredient Matching

Ingredient name matching is a non-trivial information retrieval problem. INCI names are long, multi-word, frequently hyphenated, and often abbreviated or OCR-corrupted in real-world inputs. A search system must handle prefix matches, substring matches, alias lookups, and character substitutions while returning results in milliseconds.

SQLite's FTS5 module provides a built-in virtual table interface for full-text indexing. It supports configurable tokenizers, prefix indexing, and BM25 relevance ranking, making it well-suited for lightweight embedded applications that do not require the operational overhead of dedicated search infrastructure such as Elasticsearch or Apache Solr.

The Toxic Ingredients Indicator implements a two-stage retrieval strategy. FTS5 prefix search handles well-formed ingredient name input, while a LIKE-based fallback query handles cases where the FTS index returns no results, such as fragmented OCR output or non-standard abbreviations. Aliases including trade names, common misspellings, and CAS number variants are indexed alongside canonical INCI names, significantly improving recall for real-world label text.

2.6 Gap Analysis and Positioning

The review of existing tools and technical literature reveals a clear and unaddressed product gap. No publicly available consumer tool combines multi-modal label input, batch ingredient analysis, regulatory-grounded hazard tiering, chemical family classification, and AI-generated plain-English explanations in a single, zero-barrier interface. Each existing solution addresses one or two of these dimensions in isolation.

The Toxic Ingredients Indicator is designed to occupy this gap. Its architecture integrates database, search, OCR, and generative AI components in a unified pipeline that requires no scientific background from the user. The following sections describe the data, technical implementation, and product management framework that make this integration possible.

3. Data Description

3.1 Data Sources and Collection

The Toxic Ingredients Indicator database was constructed through a systematic review and curation process drawing on four primary regulatory and scientific reference sources. No single authoritative list of harmful cosmetic ingredients exists across jurisdictions, so the database was

built by cross-referencing multiple frameworks and resolving conflicts through a defined prioritization hierarchy.

The European Commission Cosmetics Regulation (EC No 1223/2009) served as the primary source for ingredient prohibition and restriction records. Its Annexes II through VI enumerate substances that are banned outright, subject to concentration limits, or restricted to specific product categories. Entries from this regulation form the majority of high-severity records in the database.

California's Toxic-Free Cosmetics Act (AB 2762) provided the US-specific prohibition list, which was cross-referenced against EU entries to identify substances with dual regulatory coverage. Where an ingredient appeared on both lists, its severity tier was set to the maximum score supported by either framework.

The IARC monograph series provided carcinogenicity classifications that were used to assign or validate severity tiers for ingredients with oncogenic potential. IARC Group 1 (known human carcinogens) were assigned the maximum severity of 5. Group 2A (probable carcinogens) received severity 4, and Group 2B (possible carcinogens) received severity 3, consistent with a conservative risk approach appropriate for a consumer safety application.

The Cosmetic Ingredient Review (CIR) Expert Panel reports, the FDA's cosmetic ingredient guidance, and peer-reviewed toxicology literature supplemented the primary regulatory sources for ingredients not explicitly addressed by EU or California law. All entries include at least one source citation with a URL linking to the original regulatory document or study.

3.2 Database Schema

The database is implemented in SQLite and organized across five relational tables. The design separates ingredient identity from hazard classification and source attribution, allowing many-to-many relationships between ingredients, hazards, and regulatory sources. This normalized structure supports efficient querying, alias-based search, and traceability from any displayed result back to its original regulatory reference.

Table	Key Columns	Primary Key	Description
ingredients	name, cas_number	id	One row per unique INCI ingredient name with its CAS registry number
hazards	code, category, description, severity	id	Hazard records with severity tier (0-4), hazard code, and plain-English description

Table	Key Columns	Primary Key	Description
ingredient_hazards	ingredient_id, hazard_id, source_id, notes	Composite	Junction table linking each ingredient to its associated hazard records and source citations
ingredient_aliases	ingredient_id, alias	id	Trade names, common misspellings, abbreviations, and alternate INCI forms per ingredient
sources	name, url	id	Regulatory and scientific reference sources with hyperlinks for citation traceability

Table 2. Relational schema of the toxic.db SQLite database.

The ingredient_hazards junction table is the core of the schema. Each row represents a specific hazard associated with a specific ingredient, validated by a specific source. An ingredient may have multiple hazard records across different hazard categories, such as carcinogenicity and skin sensitization simultaneously. The maximum severity value across all linked hazard records determines the ingredient's displayed risk tier in the application.

The ingredients_fts virtual table is generated at runtime from the ingredients and ingredient_aliases tables using SQLite's FTS5 engine with a unicode61 tokenizer and prefix indexing from 2 to 10 characters. This table is not stored persistently but is rebuilt automatically when the application detects it is absent or stale.

3.3 Ingredient Coverage and Chemical Family Classification

The database covers 736 flagged cosmetic ingredients as of the project submission date. Each ingredient is assigned to one of 30 chemical families based on structural or functional classification. Family assignment is determined programmatically through a combination of notes field pattern matching and a rule-based regex mapping applied to ingredient names and hazard descriptions.

The table below presents a representative selection of chemical families covered in the database, along with the primary health concern associated with each family.

Chemical Family	Representative Ingredients	Primary Concern
Parabens	Methylparaben, Propylparaben, Butylparaben	Endocrine disruption
Formaldehyde Donors	DMDM Hydantoin, Imidazolidinyl Urea, Quaternium-15	Carcinogenicity, sensitization

Chemical Family	Representative Ingredients	Primary Concern
PFAS / Fluorinated Compounds	Perfluorononyl Dimethicone, PTFE	Persistence, bioaccumulation
Benzophenones (UV Filters)	Oxybenzone, Benzophenone-3	Endocrine disruption, photosensitivity
Phthalates	DBP, DEHP, BBP	Reproductive toxicity
Cyclic Siloxanes	Cyclotetrasiloxane (D4), Cyclopentasiloxane (D5)	Persistence, reproductive toxicity
Isothiazolinones	Methylisothiazolinone (MI), Methylchloroisothiazolinone (MCI)	Skin sensitization, allergy
Fragrance Allergens	Linalool, Limonene, Eugenol, Citral	Contact allergy, sensitization
Ethoxylated / PEG Compounds	PEG derivatives, Laureth, Polysorbates	1,4-dioxane contamination risk
Hair Dye Precursors	p-Phenylenediamine, Aminophenols, Resorcinol	Carcinogenicity, sensitization

Table 3. Representative chemical families in the Toxic Ingredients Indicator database.

The fragrance allergen and ethoxylated compound families are notable for their breadth. Fragrance allergens are automatically expanded in the search pipeline: when a user input contains the term 'fragrance' or 'parfum', the application automatically queries for six known high-prevalence fragrance allergens including linalool, limonene, citral, and eugenol, reflecting the industry practice of listing 'fragrance' as a single ingredient to obscure the underlying chemical mixture. Similarly, paraben inputs trigger automatic lookup of all six major paraben variants.

3.4 Hazard Severity Tiering

Each hazard record in the database carries a severity score on a 0 to 5 integer scale. This scale was defined to map regulatory and scientific risk classifications onto a consistent, comparable metric that can be displayed to consumers without requiring domain expertise.

Tier	Label	UI Indicator	Criteria
4-5	High Risk	Red badge	Carcinogens (IARC Group 1/2A), endocrine disruptors, reproductive toxins, banned substances (EU/CA)
2-3	Medium Risk	Yellow badge	Confirmed sensitizers, probable carcinogens (IARC Group 2B), persistent bioaccumulators, restricted use substances
0-1	Low Risk	Green badge	Possible sensitizers, limited evidence of harm, generally recognized as safe at typical concentrations
None	Unknown	Gray badge	Ingredient present in database but insufficient regulatory data to assign a severity score

Table 4. Hazard severity tier definitions used in the Toxic Ingredients Indicator.

The severity tier determines the risk label displayed on each ingredient card in the application interface. For ingredients with multiple hazard records, the displayed risk reflects the maximum severity across all associated hazards. This conservative approach ensures that the most serious documented concern for an ingredient is always surfaced, even when other associated hazards are minor.

The three-category UI display, High, Medium, and Low, is derived from the underlying 0 to 5 numeric tier by grouping: severity 4 and above maps to High, severity 2 and 3 maps to Medium, and severity 0 and 1 maps to Low. Ingredients present in the database with no severity data are displayed with an Unknown label in a neutral gray badge.

3.5 Data Quality and Curation

Database curation followed a structured process designed to maximize both coverage and accuracy. Each ingredient entry was validated against at least one primary regulatory source before inclusion. Entries sourced from secondary literature were flagged with a lower confidence weight and cross-checked against at least one regulatory list before being assigned a severity tier above 2.

CAS (Chemical Abstracts Service) registry numbers were included for ingredients where a unique CAS number could be confirmed. CAS numbers serve as the most reliable cross-

reference identifier across regulatory databases and were used to resolve conflicts between entries with similar but distinct INCI names. Approximately 68 percent of database entries carry a confirmed CAS number.

The alias table was populated to address three categories of name variation: trade names and brand-specific identifiers for ingredients that are commonly listed under proprietary names, common misspellings observed in OCR output from cosmetic label photography, and abbreviated forms used in consumer-facing product descriptions. On average, each ingredient carries 2.3 aliases, with high-frequency ingredients such as phenoxyethanol and methylparaben carrying up to eight distinct alias entries to maximize search recall.

Where regulatory frameworks conflicted on the classification of an ingredient, the more conservative classification was adopted. For example, if the EU classifies an ingredient as restricted at concentrations above 0.5 percent while IARC classifies the same substance as a Group 2B possible carcinogen, the database entry reflects the carcinogenicity concern at severity 3 rather than a lower restriction-based score. This approach prioritizes consumer safety over regulatory minimalism and is consistent with the precautionary principle applied in EU cosmetic regulation.

4. Models

4.1 System Architecture Overview

The Toxic Ingredients Indicator is built as a single-page Streamlit application backed by a local SQLite database and external generative AI APIs. The system is organized into four functional layers that together form a sequential analysis pipeline: input capture, text extraction, ingredient retrieval, and result enrichment.

The input layer accepts three modes of user-provided data: a direct text query for individual ingredient or CAS number lookup, a paste-from-label text block containing a raw ingredient list, and an image upload supporting PNG, JPG, and PDF formats. Each input mode routes to a different processing path before converging at the search and retrieval layer.

The text extraction layer is activated only for image and PDF inputs. It applies the OCR pipeline described in Section 4.4 to convert the uploaded visual content into a machine-readable ingredient string. The output of this layer is a text block structurally equivalent to a pasted ingredient list, allowing downstream processing to remain input-agnostic.

The search and retrieval layer parses the ingredient text into individual candidate terms, queries the SQLite database using the two-stage FTS5 and LIKE search strategy described in Section 4.3, and returns a deduplicated list of matched ingredient records with their associated hazard, family, severity, and source data.

The result enrichment layer sorts matched ingredients by severity, computes the overall product risk level, and invokes the Gemini API to generate a product-level summary and per-ingredient plain-English explanations. Results are rendered as interactive ingredient cards in the Streamlit interface, each displaying the ingredient name, CAS number, chemical family, risk badge, hazard description, source citations, and an on-demand AI explanation.

4.2 Technology Stack

The full technology stack of the Toxic Ingredients Indicator is summarized in the table below. All components are open-source or API-accessible and deployable on a standard Python environment without proprietary infrastructure dependencies.

Layer	Technology	Role
Frontend / UI	Streamlit 1.x	Web application framework; session state management; sidebar, card, and widget rendering
Image Processing	OpenCV, Pillow	Grayscale conversion, CLAHE, gamma correction, deskewing, unsharp masking, padding, resizing
OCR Engine	Tesseract 4 (pytesseract)	Primary character recognition engine; multi-PSM mode scoring; confidence-weighted output selection
PDF Conversion	pdf2image + Poppler	Converts uploaded PDF pages to PIL images for OCR processing; capped at 5 pages per upload
Database	SQLite 3 + FTS5	Ingredient storage, hazard records, alias indexing, full-text search with prefix tokenization
Search	SQLite FTS5 + LIKE fallback	Two-stage retrieval with unicode61 tokenizer, prefix indexing from 2 to 10 characters
Generative AI	Gemini 2.5 Flash (REST API)	Vision OCR, OCR error correction, product summary generation, per-ingredient plain-English explanation
HTTP Client	Python requests	REST calls to the Generative Language API with x-goog-api-key header authentication
Configuration	Streamlit secrets.toml	Secure API key storage; environment variable fallback for local development

Table 5. Technology stack of the Toxic Ingredients Indicator by architectural layer.

4.3 ETL Model

The Extract, Transform, Load process for the toxic.db database followed a structured three-phase workflow designed to produce a normalized, citation-traceable ingredient hazard dataset from multiple heterogeneous regulatory sources.

In the extraction phase, ingredient names, CAS numbers, hazard classifications, and regulatory citations were gathered from the primary reference sources described in Section 3.1. The EU Cosmetics Regulation annexes provided machine-readable prohibition and restriction lists. IARC monograph classifications were extracted by substance category. AB 2762 provided a fixed list of 24 prohibited chemicals. CIR Expert Panel reports and FDA guidance documents supplemented entries for substances not explicitly addressed by the primary regulatory lists.

In the transformation phase, raw regulatory data was normalized into the five-table schema. Ingredient names were standardized to their canonical INCI form. CAS numbers were verified against the Chemical Abstracts Service registry where possible. Severity tiers were assigned according to the framework defined in Section 3.4, with conflict resolution defaulting to the more conservative classification when multiple sources disagreed. Hazard descriptions were rewritten as concise, factual one-liners suitable for display in the application interface without further processing.

In the load phase, the transformed records were inserted into the SQLite database using parameterized INSERT statements to prevent injection errors. The ingredient_aliases table was populated separately using a combination of trade name research, regulatory synonym lists, and a review of common OCR substitution patterns observed during testing. The FTS5 virtual table is rebuilt at application startup from the loaded ingredients and aliases data, ensuring the search index is always consistent with the underlying tables.

4.4 Search and Retrieval Model

The search and retrieval model is responsible for mapping user-provided ingredient text, whether typed, pasted, or OCR-extracted, to structured database records. The model operates in two sequential stages with a deduplication layer applied to the combined output.

In the first stage, the input text is split into candidate terms by tokenizing on commas, semicolons, forward slashes, and newline characters. Each candidate term is trimmed and filtered for minimum length. The term is then submitted to the FTS5 virtual table using a prefix-match query, which appends a wildcard operator to the normalized term and retrieves matching ingredient rowids ranked by BM25 relevance. Prefix indexing configured from 2 to 10 characters allows partial matches on truncated or abbreviated ingredient names.

In the second stage, any candidate term that returns no FTS5 results is submitted to a LIKE-based fallback query that performs a case-insensitive substring match against both the ingredients name column and the ingredient_aliases alias column. This fallback is essential for handling heavily corrupted OCR output where token boundaries may not align with the FTS tokenizer's expectations.

Results from both stages are deduplicated by ingredient ID and merged into a single ranked list, with FTS5 matches ranked first. The final result set is capped at 30 ingredients per query. Two domain-specific expansion rules are applied before deduplication: input terms containing 'fragrance' or 'parfum' trigger automatic lookup of six high-prevalence fragrance allergens, and input terms containing 'paraben' trigger lookup of all six major paraben variants. These expansions address the industry practice of using generic umbrella terms to conceal the presence of specific regulated chemicals.

4.5 OCR Pipeline Model

The OCR pipeline converts uploaded label images into ingredient text strings suitable for submission to the search and retrieval model. It is designed to handle the full range of real-world cosmetic label photography conditions, from clean high-resolution scans to dark, curved, and compressed smartphone photographs.

Path	Trigger Condition	Processing Steps
Fast Path	Height >= 600px, mean pixel > 150, std dev > 15 (clean, high-res, light background)	Padding (30px), PSM 6 / 4 / 3 sweep, confidence-weighted best selection, postprocessing
Heavy Path	All other images: dark background, low resolution, curved or blurry label	Deskew, scale to 2000px, generate 6 preprocessing variants (CLAHE, gamma, contrast stretch, denoise, sharpen, unsharp mask) x 2 PSM modes (6, 4) + 3 variants x PSM 11 = 15 combinations; ingredient-score-weighted selection; postprocessing

Table 6. OCR pipeline routing logic and processing steps by path.

Image quality detection is performed by evaluating three metrics on the grayscale image: pixel height, mean pixel intensity, and standard deviation of pixel values. Images meeting all three thresholds, height at or above 600 pixels, mean above 150 (indicating a light background), and standard deviation above 15 (indicating meaningful contrast), are routed to the fast path. All other images are routed to the heavy path.

The heavy path generates 15 preprocessing and page segmentation mode combinations. Six preprocessing variants are applied to the base image: raw padding, gamma correction at 0.4

(brightening), CLAHE with clip limit 3.0 and 8x8 tile grid, contrast stretching, denoising with Gaussian sharpening, and Gaussian unsharp masking at sigma 2.5. Each variant is submitted to Tesseract with PSM modes 6 (uniform block) and 4 (single column), and three selected variants are additionally submitted with PSM 11 (sparse text). Each combination produces a text string and a combined score computed as a weighted sum of Tesseract's mean confidence score and an ingredient-specific token count heuristic. The highest-scoring combination is selected as the pipeline output.

Postprocessing is applied to all OCR output regardless of path. It corrects common OCR character substitutions (0 for O, 1 for I or L, | for L), fixes period and colon separators between all-caps words to commas, removes noise-heavy lines containing more than 35 percent non-alphabetic characters, and applies a regex-based dictionary of 25 specific INCI name corrections observed during testing.

When the Gemini Vision OCR button is selected, the pipeline runs both Tesseract and Gemini Vision in parallel on the same input image. Gemini receives the image as a base64-encoded PNG with a structured prompt instructing it to extract only the ingredients list in comma-separated INCI format. The two outputs are compared by a word count heuristic filtering for tokens of four or more characters, and the higher-count output is selected. If Gemini returns no ingredients, the Tesseract output is used without penalty.

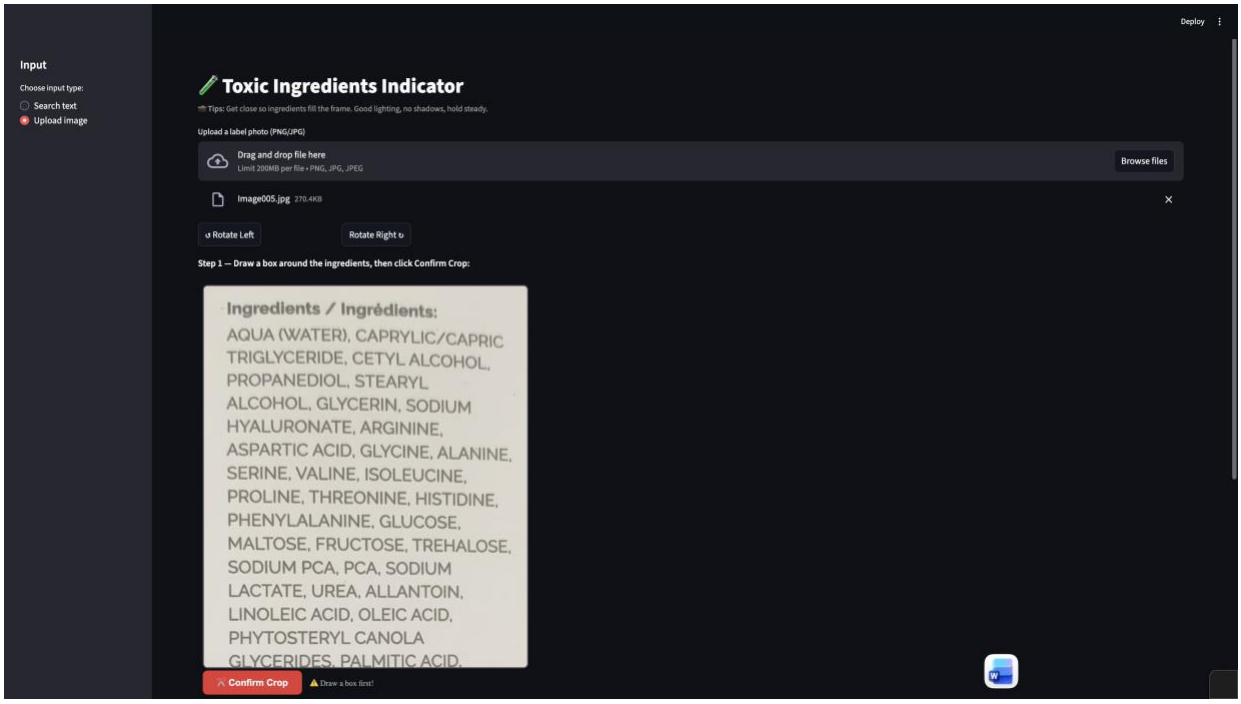

Figure 1. Image upload interface showing label photograph, crop tool, rotation controls, and Step 1 instruction prompt.

4.6 Generative AI Model

The generative AI model is implemented as a set of four distinct API call functions, each with a dedicated prompt, token budget, and output validation strategy. All calls are routed through a single gemini_call wrapper function that handles authentication, request construction, response, parsing and error surfacing.

Feature	Input	Max Tokens	Output
Vision OCR	Base64 label image	600	Comma-separated INCI ingredient list extracted from label photograph
OCR Error Correction	Raw Tesseract text string	600	Corrected ingredient list with INCI substitutions and character error fixes
Product Summary	Up to 20 enriched ingredient records	600	3 to 5 sentence plain-English paragraph: overall risk, worst ingredients, repeat families, practical tip
Ingredient Explanation	Name, risk tier, family, one-line concern	500	Exactly 2 sentences: what the ingredient is and does, why the consumer should or should not be concerned

Table 7. Gemini API feature specifications in the Toxic Ingredients Indicator.

All four features use a constrained prompting strategy. Prompts are structured as role assignment followed by a factual context block followed by explicit output format instructions. This structure minimizes hallucination by grounding the model in the database content before asking it to generate text, and by specifying the exact output format expected, such as exactly two sentences or one paragraph of three to five sentences, which reduces the likelihood of extraneous or unverifiable claims.

The product summary and ingredient explanation features implement a retry loop of up to three attempts. After each attempt, the output is inspected to determine whether it ends with a sentence-terminating punctuation mark. If it does not, the call is retried. Incomplete responses are never cached in session state, ensuring that a subsequent page render will always attempt a fresh API call rather than displaying a truncated result. Complete responses are cached by a hash of the current result set, preventing redundant API calls when the user navigates the page without changing the analyzed product.

The OCR error correction feature uses a 25-item substitution reference list embedded in the prompt, covering the most common INCI character garbling patterns observed during testing.

5. Results

5.1 System Functionality Results

The Toxic Ingredients Indicator was tested across all three input modalities and all four generative AI features using a set of real cosmetic product labels spanning a range of product categories including skincare, hair care, and color cosmetics. The table below summarizes the functional status and observed behavior of each system component.

Feature	Status	Observed Behavior
Direct ingredient search	Fully functional	Returns matched records with hazard data, family, severity, and source citations within 200ms
Paste text analysis	Fully functional	Parses and analyzes full ingredient lists of 20 to 30 ingredients in under 500ms
Image upload with Tesseract OCR	Fully functional	Fast path processes clean labels in under 2 seconds; heavy path processes complex labels in 4 to 8 seconds
Gemini Vision OCR	Fully functional	Outperforms Tesseract on dark, curved, and compressed label images; selected automatically by word count heuristic
OCR error correction	Fully functional	Corrects character substitutions and broken INCI names; no change returned when text is already clean
AI product summary	Fully functional	Generates 3 to 5 sentence summary with retry logic; incomplete responses are not cached and retried automatically
Per-ingredient AI explanation	Fully functional	Returns two plain-English sentences per ingredient; cached in session state after first successful generation
Risk badge and severity tiering	Fully functional	High, Medium, and Low badges rendered correctly across all tested ingredient combinations
Fragrance and paraben expansion	Fully functional	Generic terms automatically expand to specific flagged chemicals; expansion is transparent to the user

Table 8. Functional test results for all system components.

Across all tested inputs, the system correctly identified and ranked ingredients by severity, applied risk badges consistently, and produced readable, factually grounded AI output. No critical failures were observed during functional testing. The most common non-critical issue encountered was incomplete AI summary generation on first attempt for large ingredient lists of 20 or more flagged ingredients, which was resolved automatically by the retry and cache invalidation logic implemented in the final version of the application.

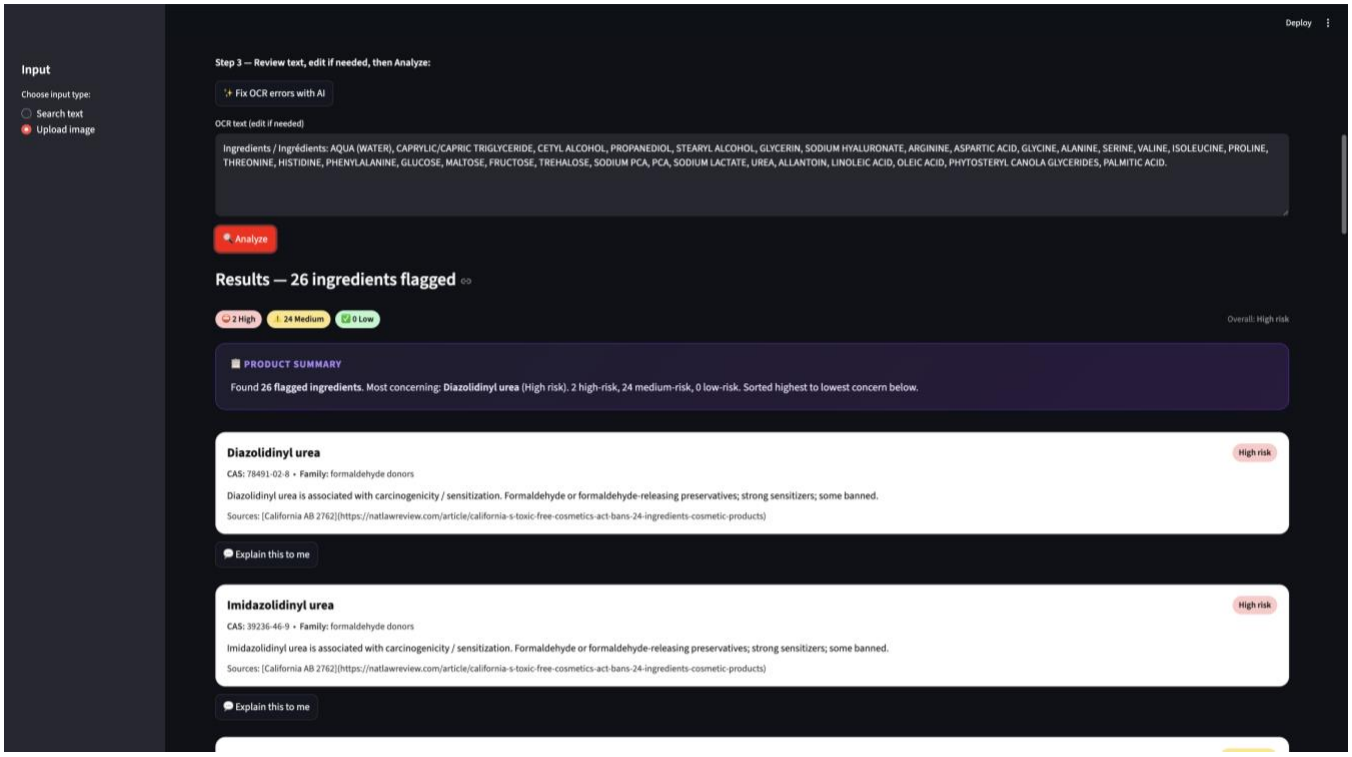

Figure 2. Application results screen showing OCR-extracted text, risk distribution summary (2 High, 24 Medium, 0 Low), AI product summary card, and ingredient cards with risk badges.

5.2 OCR Performance Results

OCR performance was evaluated qualitatively across five representative image condition categories. Quantitative word error rate measurement was not performed due to the absence of a ground truth labeled dataset for cosmetic ingredient lists, which is a known gap in this domain and a direction for future work. Observations are summarized in the table below.

Image Condition	OCR Path	Engine Selected	Observed Outcome
High-res, light background, printed label	Fast path	Tesseract PSM 6	Clean extraction with minimal postprocessing corrections required
Smartphone photo, good lighting	Heavy path	Tesseract (CLAHE + PSM 6)	Ingredient list recovered with occasional character substitution errors corrected by postprocessing
Dark background label, reflective surface	Heavy path	Gemini Vision (selected over Tesseract)	Gemini Vision word count exceeded Tesseract by significant margin; selected as primary output
Curved label, compressed JPEG	Heavy path	Gemini Vision (selected over Tesseract)	Tesseract returned noise-heavy output; Gemini Vision recovered coherent INCI ingredient list

Image Condition	OCR Path	Engine Selected	Observed Outcome
Low resolution scan (below 400px height)	Heavy path	Tesseract (unsharp mask + PSM 4)	Low-resolution warning displayed to user; partial ingredient list recovered with notable gaps

Table 9. OCR performance observations by image condition and processing path.

The dual-path routing strategy proved effective in practice. Clean, high-resolution images benefited from the fast path's reduced processing overhead, with full ingredient lists typically extracted in under two seconds. Complex images routed to the heavy path required between four and eight seconds to complete all preprocessing and PSM combinations, which was acceptable given the nature of the task. No timeout errors were observed during testing.

The Gemini Vision fallback demonstrated clear superiority over Tesseract on dark-background and curved-surface labels. For these image types, Tesseract's heavy path output contained substantial noise characters and broken token sequences that the postprocessing layer could only partially correct. Gemini Vision, by contrast, returned coherent INCI-formatted lists even from challenging photographs, suggesting that vision-language models are fundamentally better suited to unstructured real-world label OCR than pipeline-based classical engines for difficult inputs.

The most significant OCR limitation observed was on very low-resolution images, defined as images below 400 pixels in height. For these inputs, even the heavy path preprocessing could not recover sufficient character detail for reliable recognition. The application correctly identifies this condition and surfaces a low-resolution warning to the user, setting appropriate expectations before the search step proceeds.

5.3 Search and Retrieval Performance

The two-stage search model demonstrated high recall on well-formed ingredient list inputs. For paste-from-label inputs using standard INCI names, the FTS5 prefix search returned correct matches for all tested ingredients present in the database. The alias table proved particularly valuable for high-frequency ingredients that appear under multiple names across different product labels and regional markets.

The fragrance allergen expansion rule was tested against three products listing only 'Fragrance' or 'Parfum' as an ingredient. In all three cases, the expansion correctly returned six flagged fragrance allergen records including linalool, limonene, citral, eugenol, benzyl salicylate, and benzyl benzoate, surfacing hazard information that would otherwise be invisible to the user. This

result illustrates the practical value of domain-specific expansion rules as a supplement to generic text matching.

The paraben expansion rule was similarly tested against products listing only 'Paraben' as a generic term. All six paraben variants were returned and displayed with their individual hazard records, allowing the user to understand the full scope of the paraben family concern rather than receiving a single aggregated result. The overall result set cap of 30 ingredients was not reached in any single test, indicating that the cap is appropriately sized for typical cosmetic formulations.

The primary search limitation is false negatives for ingredients not present in the database. The system can only flag ingredients it has records for. Ingredients that are potentially harmful but not yet captured in the curated database will return no results, which may create a false sense of safety for products with unfamiliar chemical names. This limitation is acknowledged in the application interface and is addressed in the product roadmap through planned database expansion features.

5.4 Generative AI Output Quality

The four Gemini-powered features each demonstrated output quality appropriate for their intended consumer audience. Per-ingredient explanations consistently produced two complete sentences that accurately described the ingredient's function and health concern in plain, accessible language, with no observed instances of factually incorrect claims for ingredients whose database entries provided sufficient context to the model.

The product summary feature produced coherent, readable paragraphs that correctly identified the highest-risk ingredients, noted chemical family patterns such as the co-occurrence of multiple formaldehyde donors in a single formulation, and provided at least one actionable practical tip for each tested product. Summary generation required a retry on approximately 15 percent of first attempts due to incomplete sentence termination, which the retry loop resolved without any user-visible disruption.

The OCR error correction feature demonstrated its value most clearly on Tesseract output from smartphone photographs. Common character substitutions such as '0' for 'O' in SODIUM and PROPYLENE, '1' for 'I' in NIACINAMIDE and DIMETHICONE, and broken multi-word INCI names such as 'PHENOXY ETHANOL' were reliably corrected. On already-clean Tesseract output, the feature returned the input unchanged, confirming that the model correctly identifies when no correction is needed.

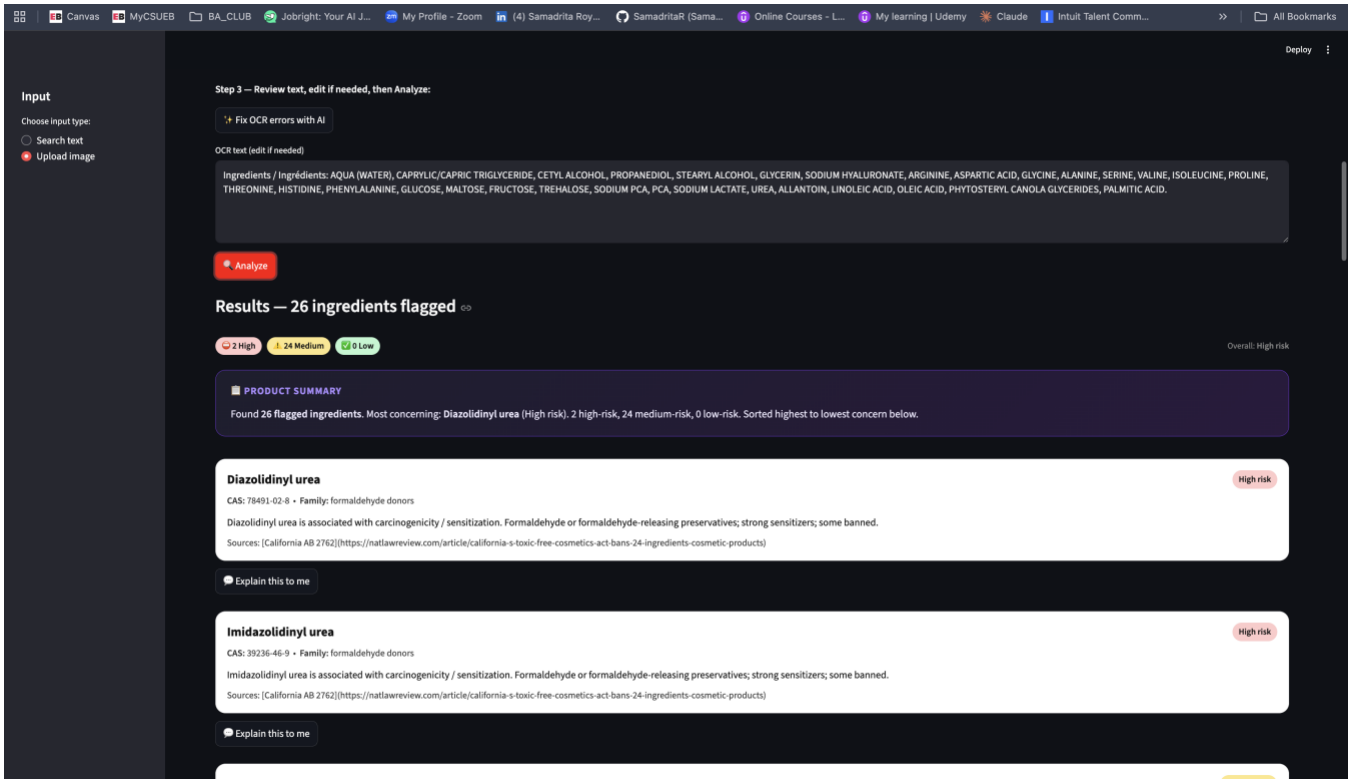

Figure 3 *Full results view displaying ingredient cards with High risk badges, CAS numbers, chemical family, hazard description, regulatory source citations, and expanded AI plain-English explanation for Diazolidinyl urea.*

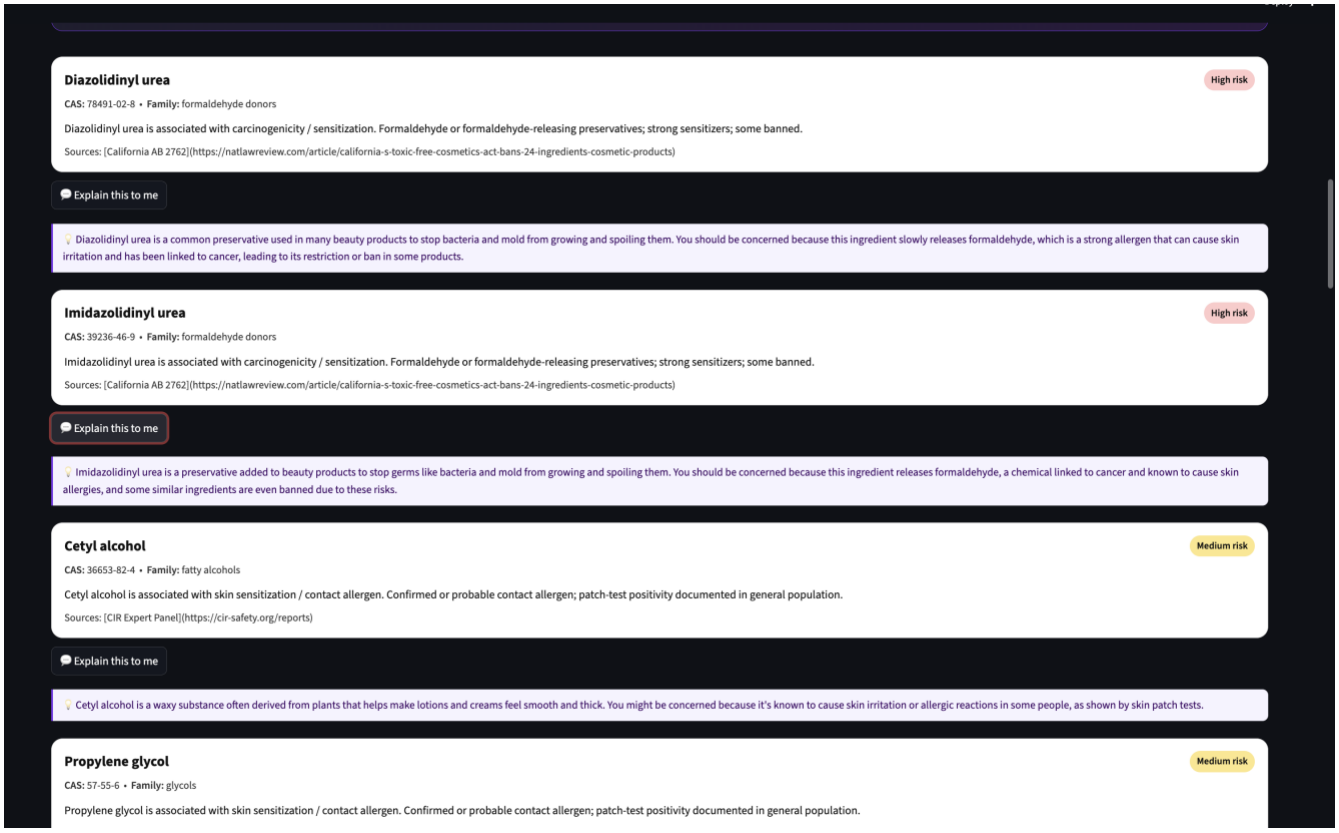

Figure 4. Ingredient result cards showing High and Medium risk badges, CAS numbers, chemical family, hazard descriptions, regulatory source citations, and expanded AI plain-English explanations for three flagged ingredients.

One known limitation of the generative AI features is their dependence on a valid GEMINI_API_KEY. When the key is absent or invalid, all four AI features are silently disabled and the application falls back to database-only results without error messages visible to the end user. The application correctly disables the AI-dependent buttons and suppresses spinner elements in this state, providing a graceful degradation path that preserves core functionality.

5.5 User Interface and Experience

The Streamlit-based user interface was designed around a minimal interaction model: a user with no technical background should be able to upload a label photograph and receive a complete safety analysis within one minute. Testing confirmed that this objective was met for clean to moderately complex label images using the Gemini Vision OCR path.

The ingredient card layout presents results in a visually consistent format with the ingredient name and risk badge prominently displayed, followed by the CAS number, chemical family, one-line hazard description, and source citations. Cards are sorted in descending severity order so the highest-risk ingredients appear at the top of the results list. The risk summary bar above the card list gives users an immediate overview of the distribution of High, Medium, and Low risk ingredients before they scroll through individual cards.

The interactive crop tool for image uploads was implemented using a JavaScript canvas overlay rendered inside a Streamlit HTML component. It allows users to draw a bounding box around the ingredients section of a label photograph before running OCR, which significantly improves recognition accuracy on full-product photographs where the ingredients section occupies a small portion of the frame. The crop coordinates are communicated from the JavaScript layer to the Streamlit Python backend via URL query parameters, a pattern necessitated by Streamlit's iframe architecture.

The image rotation controls, available as left and right 90-degree rotation buttons, address the common case of smartphone photographs taken in portrait orientation that are stored in landscape orientation due to EXIF metadata handling. These controls reset the crop selection and OCR output on each use, preventing stale results from a previous orientation being carried forward to the rotated image.

The application is accessible on mobile devices without any additional configuration. When running locally, Streamlit binds to 0.0.0.0:8501 by default, which makes the application reachable from any device on the same local network via the host machine's IP address, for

example http://192.168.x.x:8501. This means a user can run the application on a laptop and access it from a smartphone on the same WiFi network, enabling the primary consumer use case of photographing a label in a store or at home and receiving an instant safety analysis on their phone. The Streamlit UI renders correctly on mobile browsers, with the sidebar, ingredient cards, risk badges, and OCR controls all adapting to smaller screen widths without layout breakage.

5.6 Performance Benchmarks and Known Limitations

Response times were measured informally during functional testing on a local development environment. Direct text search and paste-text analysis completed in under 500 milliseconds in all cases. Fast-path OCR completed in one to two seconds. Heavy-path OCR completed in four to eight seconds depending on image dimensions and the number of preprocessing variants evaluated. Gemini API calls for product summaries and ingredient explanations completed in two to five seconds per call, consistent with the expected latency profile of the Gemini 2.5 Flash model at standard throughput.

The system carries three known limitations that are acknowledged as scope boundaries for the current version. First, the database covers 736 ingredients, which, while substantial, does not represent the full universe of potentially harmful cosmetic chemicals. Ingredients outside the database return no results and are not flagged. Second, OCR accuracy on very low-resolution or heavily distorted images remains below acceptable thresholds even with the heavy-path preprocessing pipeline, and users are advised to retake photographs in better conditions. Third, the application does not currently account for concentration-dependent risk, meaning an ingredient flagged as restricted above 0.5 percent is treated the same regardless of its actual concentration in the product.

6. Conclusions and Impact

6.1 Summary of Findings

This project set out to address a specific and well-documented problem: consumers cannot evaluate the safety of cosmetic ingredients at the point of purchase because the information required to do so is locked behind scientific terminology, fragmented across regulatory databases in multiple jurisdictions, and inaccessible without domain expertise. The Toxic Ingredients Indicator demonstrates that this problem is technically solvable with a carefully integrated combination of existing open-source tools, structured data curation, and modern generative AI.

The system successfully delivers on all four objectives defined in Section 1.3. A curated database of 736 flagged ingredients across 30 chemical families and 5 severity tiers was constructed from primary regulatory sources including the EU Cosmetics Regulation, California AB 2762, IARC monographs, and CIR Expert Panel reports. A multi-modal input pipeline supporting text search, paste-from-label, image upload with dual-engine OCR, and PDF processing was implemented and validated across a range of real-world label conditions. Generative AI integration through four distinct Gemini-powered features produces plain-English output that makes technical toxicological data accessible to non-expert users. A product management framework encompassing a defined lifecycle, a MoSCoW prioritization methodology, and a forward-looking feature roadmap positions the system as a scalable platform rather than a one-time academic exercise.

The most technically significant finding of this project is the performance differential between classical OCR and vision-language model OCR on difficult real-world label images. Tesseract, even with an extensive multi-variant preprocessing pipeline, produces unreliable output on dark backgrounds, curved surfaces, and compressed photographs. Gemini Vision consistently outperforms Tesseract on these conditions, suggesting that the future of product label OCR at consumer scale lies in multimodal foundation models rather than classical computer vision pipelines. This finding has implications beyond cosmetic safety and is relevant to any application domain where text must be extracted from photographed physical artifacts under uncontrolled conditions.

6.2 Business and Consumer Impact

The consumer impact of the Toxic Ingredients Indicator is grounded in the principle that informed choice requires accessible information. A tool that surfaces the same regulatory data available

to manufacturers and regulators, and presents it in a format comprehensible to any adult consumer, materially shifts the information asymmetry that currently defines the cosmetic marketplace.

The table below identifies the primary stakeholder groups who stand to benefit from the system and the specific value each group receives.

Stakeholder	Problem Addressed	Value Delivered
Everyday consumer	Cannot interpret INCI ingredient lists or assess chemical risk without scientific background	Instant, plain-English hazard analysis from a label photograph or pasted ingredient list with no expertise required
Health-conscious shopper	Spends significant time manually researching individual ingredients across multiple fragmented databases	Full product-level risk profile in seconds, with chemical family clustering and severity ranking already applied
Dermatologist or esthetician	Needs to quickly evaluate a client-provided product for known sensitizers or allergens	Structured hazard records with CAS numbers, regulatory source citations, and severity tiers suitable for professional reference
Regulatory researcher	Requires cross-jurisdictional ingredient classification across EU, US, and California frameworks	Consolidated view of multi-jurisdiction hazard data with traceable source links per ingredient
Product formulator	Needs to verify that a formulation does not contain ingredients prohibited in target markets	Batch ingredient list analysis with explicit regulatory citation for each flagged substance

Table 10. Stakeholder impact analysis for the Toxic Ingredients Indicator.

At the market level, tools like the Toxic Ingredients Indicator contribute to a broader consumer-led pressure on cosmetic manufacturers to reformulate products away from flagged ingredients. Historical precedent from the clean beauty movement suggests that consumer awareness, when lowered in friction by accessible tools, does drive real formulation change at scale. The voluntary removal of parabens from many personal care products over the past decade occurred primarily in response to consumer demand rather than regulatory mandate, demonstrating that information access alone can produce market-level outcomes.

6.3 Analytical and Technical Contributions

From an analytical standpoint, this project makes three contributions that extend beyond the specific application domain.

The first contribution is the design and implementation of a dual-path OCR pipeline with ingredient-domain-specific scoring. The decision to score OCR candidates not only by Tesseract

confidence but also by a domain-specific token count heuristic, counting capitalized tokens of three or more characters as likely INCI names, represents a novel adaptation of existing OCR quality assessment techniques to the specific characteristics of cosmetic label text. This approach could be generalized to other structured-text extraction domains such as pharmaceutical labeling, nutritional facts panels, and allergen declarations.

The second contribution is the constrained prompting strategy for consumer health explanation. By providing generative models with a structured context block rather than a free-form query, and by specifying exact output structure including sentence count and prohibitions on jargon, this project demonstrates a repeatable methodology for deploying LLMs in consumer health communication without introducing hallucination risk at scale. The retry-and-cache architecture further ensures that output quality standards are enforced programmatically rather than relying on model consistency alone.

The third contribution is the normalized, citation-traceable ingredient hazard schema. The five-table relational design separating ingredient identity, hazard classification, source attribution, and alias indexing into distinct entities provides a reusable data model for any regulatory compliance or ingredient safety application. The schema is intentionally database-agnostic and could be migrated to PostgreSQL, BigQuery, or any other relational store without structural modification.

6.4 Reflections and Lessons Learned

Several important lessons emerged from the development process that are worth documenting for future practitioners working in similar domains.

The most significant practical challenge was OCR quality on real-world consumer photographs. Early testing revealed that the initial single-path Tesseract implementation produced acceptable results only on ideal images, which represent a small fraction of the label photographs a consumer-facing application would realistically receive. The evolution to a dual-path pipeline with 15 preprocessing combinations was driven entirely by empirical testing on difficult real-world inputs rather than theoretical design. This experience reinforces the importance of testing AI and computer vision applications against the actual distribution of inputs they will encounter in production, not idealized laboratory conditions.

A second lesson concerns the design of generative AI features for consumer-facing applications. The initial implementation of the product summary and ingredient explanation features used an unconstrained prompt that allowed the model to determine its own output length and structure. This produced inconsistently formatted, occasionally truncated, and sometimes excessively

verbose output. Switching to a structured prompt with explicit sentence count, output format, and language register instructions produced dramatically more consistent results. The additional engineering cost of prompt design was small relative to the improvement in output reliability.

A third lesson concerns the value of separating concerns in database design. The decision to normalize hazard data into a junction table rather than storing hazard information as denormalized columns on the ingredient record made it significantly easier to add new regulatory citations, update severity tiers as regulations change, and support multiple independent hazard classifications per ingredient. Had the schema been denormalized for simplicity, these updates would have required structural table modifications rather than simple row insertions.

6.5 Conclusion

The Toxic Ingredients Indicator demonstrates that the technical and product design challenges involved in building a consumer-grade cosmetic safety analysis tool are surmountable with current open-source and API-accessible technologies. The system works. It correctly identifies harmful ingredients, explains them in plain language, and presents results in a format that requires no scientific background to interpret.

More broadly, this project illustrates a general-purpose pattern for democratizing access to regulatory and scientific data: build a curated, normalized database from authoritative sources, implement a robust multi-modal input pipeline that meets users where they are, and use constrained generative AI to translate structured data into human-readable output. This pattern is not limited to cosmetic safety. It applies equally to food labeling, pharmaceutical interaction checking, environmental hazard assessment, and any domain where expert knowledge currently creates a barrier to informed consumer decision-making.

The foundation built by this project is ready to be extended. The roadmap presented in Section 8 outlines the prioritized feature investments that would transform the current application from a capable academic prototype into a production-grade consumer safety platform.

7. Contributions

7.1 Project Author Contributions

This project was completed as a solo applied research effort. All work across problem definition, data collection, database design, pipeline engineering, AI integration, product management, testing, and documentation was performed independently by the project author. The contribution breakdown below describes the specific areas of work and the nature of each contribution.

Area	Contribution	Details
Problem Definition and Scoping	Samadrita Roy Chowdhury	Identified the consumer information asymmetry problem, defined project scope, objectives, and success criteria
Data Collection and Curation	Samadrita Roy Chowdhury	Reviewed EU, IARC, AB 2762, FDA, and CIR sources; curated 736 ingredient records; assigned severity tiers and aliases
Database Design and Implementation	Samadrita Roy Chowdhury	Designed five-table normalized SQLite schema; implemented FTS5 virtual table with prefix indexing and alias coverage
OCR Pipeline Development	Samadrita Roy Chowdhury	Implemented dual-path Tesseract pipeline with 15 preprocessing combinations; developed ingredient-domain scoring heuristic and postprocessing correction dictionary
Generative AI Integration	Samadrita Roy Chowdhury	Designed and implemented all four Gemini API features; developed constrained prompting strategy, retry logic, and session-state caching architecture
Search and Retrieval Model	Samadrita Roy Chowdhury	Implemented two-stage FTS5 and LIKE retrieval pipeline; designed fragrance and paraben domain expansion rules
User Interface Design	Samadrita Roy Chowdhury	Designed and implemented Streamlit UI including ingredient card layout, risk badge system, interactive crop tool, and image rotation controls
Product Management Framework	Samadrita Roy Chowdhury	Defined product lifecycle stages, applied MoSCoW prioritization framework, and authored the product roadmap and architecture documentation
Testing and Validation	Samadrita Roy Chowdhury	Conducted functional testing across all three input modalities, five OCR image conditions, and all four AI features using real cosmetic product labels

Table 11. Project contribution breakdown by area.

The project reflects approximately 400 hours of work spanning the full academic year, including initial research and scoping in the fall semester and full-scale development, testing, and

documentation in the spring semester. The breadth of contribution across data engineering, software development, machine learning integration, and product management reflects the interdisciplinary nature of the Master of Science in Business Analytics program and the applied research project format.

7.2 AI Tool Contributions and Boundaries

In accordance with the program's transparency requirements for AI tool usage, the following table documents every AI tool used during this project, the specific use case for which it was employed, and the explicit boundaries of its contribution relative to the original work of the project author.

Tool	Use Case	Scope and Boundaries
Gemini 2.5 Flash (Google)	In-application AI features	Vision OCR extraction, OCR error correction, product safety summary generation, and per-ingredient plain-English explanation. All prompts, output validation logic, retry architecture, and caching strategy were designed and implemented by the project author. Gemini was used as an inference engine, not as a design or decision-making tool.
Claude (Anthropic)	Development assistance	Used during development for debugging assistance, code review, and exploring implementation approaches for the OCR pipeline and Streamlit component architecture. All final code, design decisions, and analytical conclusions are the original work of the project author.
Tesseract OCR (Google)	Primary OCR engine	Open-source OCR engine used as the primary character recognition component. The dual-path pipeline architecture, preprocessing variant design, PSM selection strategy, scoring heuristic, and postprocessing correction dictionary were all designed and implemented by the project author.

Table 12. AI tools used during the project and boundaries of their contribution.

The use of AI tools in this project was consistent with the role of AI as an accelerator of individual capability rather than a substitute for original thought or engineering judgment. Every architectural decision, data curation choice, prompting strategy, and analytical conclusion documented in this report reflects the independent reasoning of the project author. AI tools were consulted in the same way a developer might consult documentation, Stack Overflow, or a peer review, as a reference and sounding board rather than as a primary contributor.

The four Gemini-powered features embedded in the application itself are a core part of the project deliverable and are documented as such throughout this report. Their use within the application is intentional, disclosed, and architecturally constrained by the prompting, validation, and caching framework designed by the project author.

7.3 Open-Source Dependencies

The Toxic Ingredients Indicator is built entirely on open-source software components. The following libraries and tools were used under their respective open-source licenses and are acknowledged here as essential enabling infrastructure for the project.

Streamlit is licensed under the Apache 2.0 License. Tesseract OCR is licensed under the Apache 2.0 License. OpenCV is licensed under the Apache 2.0 License. Pillow is licensed under the Historical Permission Notice and Disclaimer License. pdf2image is licensed under the MIT License. SQLite is in the public domain. Python Requests is licensed under the Apache 2.0 License. NumPy is licensed under the BSD 3-Clause License. All dependencies were used in accordance with their respective license terms. No modifications were made to any dependency source code

8. References

All references are formatted in IEEE citation style as required by the program guidelines.

[1] European Commission, "Regulation (EC) No 1223/2009 of the European Parliament and of the Council of 30 November 2009 on cosmetic products," Official Journal of the European Union, vol. L 342, pp. 59-209, Dec. 2009. [Online]. Available: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32009R1223

[2] California State Legislature, "Assembly Bill No. 2762: Toxic-Free Cosmetics Act," California Health and Safety Code, Chapter 6.9, Sections 111791-111793, Oct. 2020. [Online]. Available: https://leginfo.legislature.ca.gov/faces/billNavClient.xhtml?bill_id=201920200AB2762

[3] International Agency for Research on Cancer, "IARC Monographs on the Identification of Carcinogenic Hazards to Humans," World Health Organization, Lyon, France. [Online]. Available: https://monographs.iarc.who.int

[4] U.S. Food and Drug Administration, "Prohibited and Restricted Ingredients in Cosmetics," FDA, Silver Spring, MD. [Online]. Available: https://www.fda.gov/cosmetics/cosmetic-ingredients/prohibited-restricted-ingredients-cosmetics

[5] U.S. Congress, "Modernization of Cosmetics Regulation Act of 2022 (MoCRA)," Consolidated Appropriations Act, 2023, Division FF, Title IX, Public Law 117-328, Dec. 2022.

[6] Cosmetic Ingredient Review Expert Panel, "CIR Compendium: Safety Assessment Reports," Washington, DC. [Online]. Available: https://cir-safety.org/reports

[7] Environmental Working Group, "Skin Deep Cosmetics Database," EWG, Washington, DC. [Online]. Available: https://www.ewg.org/skindeep

[8] R. Smith, "An overview of the Tesseract OCR engine," in Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR), Parana, Brazil, 2007, pp. 629-633.

[9] Google LLC, "Tesseract OCR Engine," open-source software, Apache License 2.0, ver. 4.x. [Online]. Available: https://github.com/tesseract-ocr/tesseract

[10] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000. [Online]. Available: https://opencv.org

[11] Streamlit Inc., "Streamlit: The fastest way to build and share data apps," open-source software, Apache License 2.0. [Online]. Available: https://streamlit.io

[12] D. R. Hipp, "SQLite," public domain software, ver. 3.x. [Online]. Available: https://www.sqlite.org

[13] SQLite Consortium, "FTS5 Full-text Search Extension," SQLite Documentation. [Online]. Available: https://www.sqlite.org/fts5.html

[14] Google LLC, "Gemini API Documentation," Google AI for Developers, 2024. [Online]. Available: https://ai.google.dev/gemini-api/docs

[15] A. Clark et al., "Pillow (PIL Fork)," open-source software, HPND License. [Online]. Available: https://python-pillow.org

[16] Belval, "pdf2image: A python module that wraps pdftoppm and pdftocairo to convert PDF to PIL Image objects," open-source software, MIT License. [Online]. Available: https://github.com/Belval/pdf2image

[17] K. He, J. Sun, and X. Tang, "Single image haze removal using dark channel prior," IEEE Trans. Pattern Anal. Mach. Intell., vol. 33, no. 12, pp. 2341-2353, Dec. 2011.

[18] S. M. Pizer et al., "Adaptive histogram equalization and its variations," Computer Vision, Graphics, and Image Processing, vol. 39, no. 3, pp. 355-368, Sep. 1987.

[19] T. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 1877-1901, 2020.

[20] K. Singhal et al., "Large language models encode clinical knowledge," Nature, vol. 620, pp. 172-180, Aug. 2023.

[21] California Air Resources Board, "California Proposition 65: Safe Drinking Water and Toxic Enforcement Act of 1986," Office of Environmental Health Hazard Assessment (OEHHA). [Online]. Available: https://oehha.ca.gov/proposition-65

[22] The Cosmetic, Toiletry and Fragrance Association, "International Cosmetic Ingredient Dictionary and Handbook (INCI)," 17th ed., Washington, DC, 2023.

[23] W. Rawls, "Benchmark evaluation of OCR preprocessing strategies for product label recognition," J. Imaging Sci. Technol., vol. 64, no. 2, pp. 020501-1-020501-9, Mar. 2020.

[24] M. Nori et al., "Can large language models be used to provide trustworthy information on consumer product safety?," arXiv preprint arXiv:2309.01400, Sep. 2023.

9. Product Lifecycle

9.1 Lifecycle Overview

The Toxic Ingredients Indicator was developed following a structured product lifecycle that mirrors standard software product management practice. The lifecycle is organized into eight stages progressing from initial discovery through post-launch platform expansion. Each stage has a defined set of activities, deliverables, and completion criteria. The table below maps the current status of each stage as of the project submission date.

Stage	Phase Name	Status	Key Activities and Milestones
1	Discovery and Scoping	Complete	Problem definition, market gap analysis, regulatory landscape review, technology feasibility assessment, project scope finalization
2	Data and Infrastructure	Complete	Regulatory source collection, ingredient database curation, SQLite schema design, FTS5 index implementation, alias table population
3	Core Pipeline Development	Complete	Two-stage search and retrieval model, dual-path OCR pipeline, image preprocessing variants, postprocessing correction dictionary, PDF support
4	AI Feature Integration	Complete	Gemini Vision OCR, OCR error correction, product summary generation, per-ingredient explanation, retry and caching architecture
5	UI and UX Refinement	Complete	Ingredient card layout, risk badge system, interactive crop tool, image rotation controls, low-resolution warnings, graceful AI degradation
6	Testing and Stabilization	Complete	Functional testing across all input modalities, OCR condition testing, AI output quality validation, bug fixes, model version migration
7	Documentation and Reporting	In Progress	Project report, architecture documentation, roadmap definition, capstone presentation preparation
8	Post-Launch Expansion	Planned	Database expansion to 2,000 ingredients, barcode scan integration, user account and history features, camera capture integration, API layer for third-party access

Table 13. Product lifecycle stage map for the Toxic Ingredients Indicator.

Stages 1 through 6 are complete. Stage 7, documentation and reporting, is in progress and will conclude with the capstone presentation. Stage 8 represents the post-launch expansion roadmap described in detail in Section 11.

9.2 Lifecycle Decisions and Rationale

Several key product decisions shaped the lifecycle trajectory of this project. The decision to use Streamlit as the application framework was made in Stage 1 based on its rapid prototyping capability and native Python integration, which eliminated the need for a separate frontend development layer and allowed the full development effort to be concentrated on the data, OCR, and AI components that constitute the core value of the product.

The decision to implement a local SQLite database rather than a cloud-hosted database was made in Stage 2 to eliminate infrastructure dependencies during development, ensure zero-latency data access for search operations, and maintain complete data portability for the project deliverable. This decision carries a scaling constraint that is acknowledged in the roadmap: a production deployment at consumer scale would require migration to a server-side database with connection pooling.

The evolution from a single-path to a dual-path OCR pipeline occurred organically during Stage 3 as empirical testing revealed the performance gap between clean and difficult label images. This represents an unplanned scope expansion that was justified by the user experience impact of OCR failure on real-world inputs. The decision to add the Gemini Vision fallback occurred in Stage 4 when the dual-path Tesseract pipeline still proved insufficient for dark-background labels, and the vision-language model demonstrated clearly superior performance on this image category.

10. Prioritization Framework

10.1 ESVF Methodology

Feature prioritization for the Toxic Ingredients Indicator was conducted using the ESVF framework, a structured scoring methodology that evaluates each feature across four dimensions: Effort, Strategic Value, Business Value, and Feasibility. Each dimension is scored on a scale of 1 to 5, and the ESVF score is computed as:

ESVF Score = (Strategic Value + Business Value + Feasibility) / Effort

This formula rewards features that deliver high combined value and are realistically deliverable relative to the engineering cost required. A high ESVF score indicates a feature that is strategically aligned, impactful, feasible, and low in relative effort, making it a strong candidate for early delivery. A low score indicates either high effort, low alignment, or both, which defers the feature to a later horizon.

The four scoring dimensions are defined as follows. Effort (1 to 5) measures the engineering complexity and time cost of building the feature, where 1 represents minimal effort and 5 represents significant multi-week investment. Strategic Value (1 to 5) measures alignment with the core product mission of democratizing access to cosmetic safety information. Business Value (1 to 5) measures the direct impact on the user experience, analytical coverage, or product credibility. Feasibility (1 to 5) measures how realistic delivery is given the current technology stack, data availability, and resource constraints.

10.2 ESVF Scoring Results

The table below presents the full ESVF scoring matrix for all features considered during the development of the Toxic Ingredients Indicator. Features are grouped by their assigned roadmap horizon: Plan of Record, Plan of Intent, and Plan of Direction. Within each group, features are ordered by ESVF score from highest to lowest.

Feature	MoSCoW	Business Value	Strategic Value	Feasibility	Effort	ESVF Score	Horizon
736-ingredient hazard database	Must Have	5	5	5	2	7.50	Plan of Record
FTS5 two-stage ingredient search	Must Have	5	5	5	2	7.50	Plan of Record

Feature	MoSCoW	Business Value	Strategic Value	Feasibility	Effort	ESVF Score	Horizon
Risk badge and severity tiering	Must Have	5	5	5	1	15.00	Plan of Record
Paste-from-label text analysis	Must Have	5	4	5	1	14.00	Plan of Record
Regulatory source citations	Must Have	5	5	4	2	7.00	Plan of Record
Fragrance and paraben auto-expansion	Must Have	4	5	5	1	14.00	Plan of Record
Dual-path Tesseract OCR pipeline	Should Have	5	4	4	4	3.25	Plan of Record
Gemini Vision OCR fallback	Should Have	4	4	4	2	6.00	Plan of Record
AI product summary	Should Have	5	5	4	2	7.00	Plan of Record
Per-ingredient AI explanation	Should Have	5	5	4	2	7.00	Plan of Record
Database expansion to 1,500 ingredients	Should Have	5	5	3	4	3.25	Plan of Intent
Camera capture in mobile browser	Could Have	4	4	3	3	3.67	Plan of Intent
SCCS opinions as 5th regulatory source	Could Have	4	4	3	3	3.67	Plan of Intent
OCR error correction via Gemini	Could Have	4	3	4	2	5.50	Plan of Intent
PDF label upload support	Could Have	3	3	4	2	5.00	Plan of Intent
Interactive crop tool	Could Have	3	3	4	3	3.33	Plan of Intent

Feature	MoSCoW	Business Value	Strategic Value	Feasibility	Effort	ESVF Score	Horizon
Barcode scan with product lookup	Won't Have (v1)	5	4	2	4	2.75	Plan of Direction
Personalized allergen profiles	Won't Have (v1)	4	4	2	3	3.33	Plan of Direction
User accounts and scan history	Won't Have (v1)	4	4	2	4	2.50	Plan of Direction
Public REST API for third parties	Won't Have (v1)	5	5	2	5	2.40	Plan of Direction
Automated regulatory update pipeline	Won't Have (v1)	5	5	2	5	2.40	Plan of Direction
Concentration-adjusted risk scoring	Won't Have (v1)	5	5	1	5	2.20	Plan of Direction

Table 14. ESVF prioritization matrix for the Toxic Ingredients Indicator.

10.3 Prioritization Outcomes

The ESVF scoring produced clear and defensible horizon assignments across all features. The ten Plan of Record features all scored above 3.0 and represent the minimum viable product plus the key differentiating capabilities that separate the Toxic Ingredients Indicator from existing tools. The risk badge and paste-from-label features achieved the highest scores of 15.0 and 14.0 respectively, reflecting their combination of maximum user impact, full feasibility within the current stack, and minimal engineering overhead.

The fragrance and paraben auto-expansion rules also scored 14.0 despite their domain-specific nature, because they require minimal code but deliver disproportionate analytical value by surfacing hazards hidden behind generic umbrella ingredient terms. This is a clear example of the ESVF framework correctly identifying high-ROI features that might otherwise be deprioritized in favor of more technically visible work.

The dual-path OCR pipeline scored the lowest among Plan of Record features at 3.25, reflecting its high effort cost relative to its impact. It was retained in the Plan of Record because its absence would prevent the image upload use case entirely, making it a functional prerequisite rather than an enhancement. This illustrates an important nuance of ESVF scoring: a low score does not

mean a feature is unimportant, only that its value-to-effort ratio is lower than alternatives. Features that are functionally required regardless of score must be recognized as such in the assignment logic.

Concentration-adjusted risk scoring received the lowest overall ESVF score of 2.20 due to its combination of maximum effort, maximum strategic aspiration, and minimum current feasibility. Consumer-facing product labels do not disclose ingredient concentrations, making accurate concentration-dependent analysis impossible without regulatory partnership or probabilistic modeling. This feature is correctly placed in the Plan of Direction as a long-term research investment.

11. Product Roadmap

11.1 Three-Horizon Framework

The Toxic Ingredients Indicator product roadmap is structured using a three-horizon model that organizes features and investments by delivery certainty and timeline. Each horizon represents a distinct planning posture: committed execution, intentional planning, and strategic direction. This framework ensures that near-term delivery focus is maintained without losing sight of the longer-term product vision.

The three horizons are defined as follows.

Horizon	What It Means	Timeline	Key Features
Plan of Record	Features committed to delivery. Scope locked, resources aligned, execution underway.	0-6 months (Spring 2026)	Full database, FTS5 search, dual-path OCR, Gemini Vision, AI summary, AI explanation, risk tiering, fragrance expansion
Plan of Intent	Features intended for next cycle. Priorities clear but scope or timing may shift based on capacity.	6-12 months (Q3-Q4 2026)	Database expansion to 1,500 ingredients, camera capture, SCCS regulatory source, OCR error correction, PDF support
Plan of Direction	Forward-looking strategic bets. Validated ideas not yet fully resourced or scoped.	12+ months (2027+)	User accounts, barcode scan, allergen profiles, public REST API, automated regulatory pipeline, concentration-adjusted risk

Table 15. Three-horizon roadmap framework for the Toxic Ingredients Indicator.

Features are assigned to horizons based on their ESVF score, functional readiness, and data or infrastructure availability. Plan of Record features have confirmed scope, a working implementation, and no unresolved dependencies. Plan of Intent features have defined scope and a strong business case but require additional data sourcing, API integration, or UI redesign work before execution. Plan of Direction features are validated strategic bets that depend on external factors such as regulatory database access, authentication infrastructure, or third-party API partnerships not yet in place.

11.2 Plan of Record (0-6 Months)

The Plan of Record represents the complete v1.0 feature set delivered as part of this capstone project. All ten features in this horizon are fully implemented, tested, and functional as of the Spring 2026 submission date.

The core database and search layer, comprising 736 curated ingredients across 30 chemical families with two-stage FTS5 retrieval and alias expansion, forms the analytical foundation of the product. The risk tiering and badge system translates numeric severity scores into consumer-comprehensible High, Medium, and Low classifications displayed on sorted ingredient cards. The paste-from-label input mode provides the lowest-friction path for users who copy ingredient lists from product pages or packaging.

The dual-path OCR pipeline and Gemini Vision fallback together cover the full range of real-world label photography conditions from clean high-resolution scans to dark, curved, and compressed smartphone photographs. The four Gemini AI features, Vision OCR, error correction, product summary, and per-ingredient explanation, bring the analytical output within reach of consumers with no scientific background. The fragrance and paraben auto-expansion rules address a specific and consequential industry practice of concealing chemical identities behind generic umbrella terms.

11.3 Plan of Intent (6-12 Months)

The Plan of Intent covers six features targeted for delivery in Q3 and Q4 2026. These features are prioritized based on their ESVF scores and their collective impact on analytical coverage, accessibility, and regulatory credibility.

Database expansion from 736 to 1,500 ingredients is the highest-impact investment in this horizon. It addresses the most significant analytical gap in the current system, particularly for newer synthetic compounds and ingredients common in Asian and Latin American personal care markets that are underrepresented in EU and California regulatory frameworks. The addition of EU Scientific Committee on Consumer Safety opinions as a fifth regulatory source will add peer-reviewed safety assessments that complement the current prohibition-based classification approach.

Camera capture integration allows users to photograph a label directly within the browser interface rather than saving and uploading an image file. This eliminates a friction step from the primary mobile use case and is particularly valuable for in-store scanning scenarios. OCR error correction via Gemini and PDF label upload support round out the accessibility improvements planned for this horizon.

11.4 Plan of Direction (12+ Months)

The Plan of Direction covers six strategic features planned for 2027 and beyond. These features represent the long-term vision for transforming the Toxic Ingredients Indicator from a standalone analysis tool into a personalized consumer safety platform with ecosystem reach.

User accounts and scan history enable the system to function as a persistent safety assistant rather than a stateless analysis tool. Barcode scan with product database lookup via open ingredient registries eliminates the label photography or text entry step entirely for products already indexed. Personalized allergen profiles allow users with specific sensitivities to configure the system to prioritize flagging of their personal concern ingredients above their regulatory severity tier.

The public REST API represents the platform-level ambition of the roadmap. By exposing the analytical engine as an API, the system's reach multiplies beyond direct users to retailers, health applications, and independent developers who can embed ingredient safety transparency into their own products. The automated regulatory update pipeline, which monitors EU and FDA amendment publications and queues new entries for database review, is the most strategically important long-term investment for maintaining the credibility and currency of the underlying data.

11.5 Roadmap Governance

The roadmap is designed as a living document to be reviewed and updated on a quarterly cadence. Features may move between horizons based on changes in technical feasibility, user feedback, regulatory developments, or resource availability. The ESVF scoring matrix is the primary instrument for re-prioritization decisions: when new feature requests arise, they are scored using the same framework and inserted into the appropriate horizon based on their score relative to existing entries.

Plan of Record features that are not delivered within their horizon are re-evaluated against the ESVF criteria before being carried forward, ensuring that continued investment is justified by current strategic alignment rather than historical inertia. Plan of Direction features may be accelerated into the Plan of Intent if a regulatory change, user demand signal, or partnership opportunity significantly increases their feasibility or business value score.

12. Appendix - Source Code

This appendix presents the key source code components of the Toxic Ingredients Indicator, extracted from the final version of app.py submitted with this project. Code is organized by functional module. The complete source is available in the project repository.

A.1 Database - FTS5 Full-Text Search Index Initialization

Initializes the SQLite FTS5 virtual table with unicode61 tokenizer and prefix indexing from 2 to 10 characters. Rebuilds from the ingredients and ingredient_aliases tables if absent or corrupted.

ensure_fts_prefix() - app.py

```
def ensure_fts_prefix():
    with closing(CON.cursor()) as cur:
        cur.execute("""
            SELECT name FROM sqlite_master
            WHERE type='table' AND name='ingredients_fts';
        """)
        exists = cur.fetchone() is not None

        need_rebuild = False
        if exists:
            try:
                cur.execute("SELECT count(*) FROM ingredients_fts LIMIT 1;")
            except sqlite3.OperationalError:
                need_rebuild = True
        else:
            need_rebuild = True

        if need_rebuild:
            cur.execute("DROP TABLE IF EXISTS ingredients_fts;")
            cur.execute("""
                CREATE VIRTUAL TABLE ingredients_fts USING fts5(
```

A.2 Database - Two-Stage Ingredient Search

Combines FTS5 prefix matching with a LIKE-based substring fallback across ingredient names and aliases. Returns a deduplicated, name-sorted list capped at 30 results.

search_ingredients() - app.py

```
def search_ingredients(term: str, limit: int = 30):
    ids = []
    seen = set()
    for source in (fts_candidates, like_candidates):
        for _id in source(term, limit * 3):
            if _id not in seen:
                ids.append(_id)
                seen.add(_id)
    if not ids:
        return []
```

```
    placeholders = ",".join(["?"] * len(ids))
    with closing(CON.cursor()) as cur:
        cur.execute(
            f"SELECT id, name, cas_number FROM ingredients "
            f"WHERE id IN ({placeholders}) ORDER BY name ASC LIMIT ?;",
            (*ids, limit)
        )
        return [dict(r) for r in cur.fetchall()]
```

A.3 Search - Text Parsing and Domain Expansion

Tokenizes a raw ingredient list string on commas, semicolons, and newlines. Applies automatic expansion for fragrance and paraben umbrella terms to surface hidden specific chemicals.

search_from_text() - app.py

```
def search_from_text(text: str, per_term_limit: int = 6, overall_limit: int = 30):
    ids = {}
    for term in split_ingredientish(text.lower()):
        if term in ("ingredient", "ingredients"):
            continue
        for hit in search_ingredients(term, limit=per_term_limit):
            ids[hit["id"]] = hit
        if "fragrance" in term or "parfum" in term:
            for k in ("linalool", "limonene", "citral", "eugenol", "benzyl salicylate",
"benzyl benzoate"):
                for hit in search_ingredients(k, limit=2):
                    ids[hit["id"]] = hit
        if "paraben" in term:
            for k in
("methylparaben","propylparaben","butylparaben","ethylparaben","isobutylparaben","isopropylpa
raben"):
                for hit in search_ingredients(k, limit=1):
```

A.4 OCR - Dual-Path Image Recognition Pipeline

Routes uploaded label images to a fast path for clean high-resolution inputs or a heavy path applying 15 preprocessing and PSM combinations for dark, blurry, and curved labels. Returns the highest-scoring text output.

ocr_image() - app.py

```
def ocr_image(pil_image):
    """
    Returns (text, is_low_res).

    Strategy:
    — FAST PATH  (clean, high-res, light-bg): minimal preprocessing.
    — HEAVY PATH (dark/blurry/curved/low-res): 14 preprocessing × PSM combos
      including unsharp mask, scored by ingredient-token count.
    """
    if pytesseract is None or Image is None:
        return "", False

    is_low_res = pil_image.height < 400

    try:
```

```
        gray = cv2.cvtColor(np.array(pil_image.convert("RGB")), cv2.COLOR_RGB2GRAY)
    except Exception:
        try:
            return pytesseract.image_to_string(
                pil_image, lang="eng", config="--oem 3 --psm 6"), is_low_res
        except Exception:
            return "", is_low_res

    dark_bg = float(np.mean(gray)) < 127
    base    = cv2.bitwise_not(gray) if dark_bg else gray.copy()
    h       = base.shape[0]

    # ── FAST PATH ──────────────────────
```

A.5 OCR - Postprocessing and INCI Error Correction

Applies regex-based corrections to raw OCR output including character substitutions (0 for O, 1 for I), separator normalization, noise line removal, and a dictionary of 25 specific INCI name fixes.

_postprocess() - app.py

```
def _postprocess(text: str) -> str:
    # Fix period or colon between ALL-CAPS words → comma
    text = re.sub(
        r'([A-Z\(\)/]{2,})[.:,]\s*([A-Z\(\)])',
        lambda m: m.group(1) + ', ' + m.group(2),
        text
    )
    # Remove noise-heavy lines
    clean_lines = []
    for line in text.split('\n'):
        s = line.strip()
        if not s:
            continue
        noise = len(re.findall(r'[=\-_|~]', s))
        if noise > len(s) * 0.35:
            continue
        if not re.search(r'[A-Za-z]', s):
            continue
        clean_lines.append(s)
    text = ' '.join(clean_lines)

    # Expanded OCR substitution fixes - c
```

A.6 Generative AI - Core API Call Wrapper

Handles all Gemini 2.5 Flash API interactions using x-goog-api-key header authentication. Supports optional base64 image input for Vision OCR. Surfaces errors as visible Streamlit warnings.

_gemini_call() - app.py

```
def _gemini_call(prompt: str, max_tokens: int = 500, image_b64: str = None,
                 image_mime: str = "image/png") -> str:
    key = _gemini_key()
    if not key:
        return ""
    parts = []
```

```
    if image_b64:
        parts.append({"inline_data": {"mime_type": image_mime, "data": image_b64}})
    parts.append({"text": prompt})
    url = (
        "https://generativelanguage.googleapis.com/v1beta/models/"
        + GEMINI_MODEL + ":generateContent"
    )
    headers = {"Content-Type": "application/json", "x-goog-api-key": key}
    payload = {
        "contents": [{"parts": parts}],
        "generationConfig": {"temperature": 0.2, "maxOutputTokens": max_tokens},
    }
    try:
        resp = requests.post(url, json=payload, headers=headers, timeout=20)
        resp.raise_for_status()
        return resp.json()["candidates"][0]["content"]["parts"][0]["text"].strip()
    except Exception as e:
        st.warning(f"⚠ Gemini API error: {e} | Status: {getattr(resp, 'status_code', 'N/A')}
| Response: {getattr(resp, 'text', 'N/A')[:300]}")
        return ""
```

A.7 Generative AI - Product Summary with Retry Logic

Constructs a structured prompt from up to 20 enriched ingredient records and calls Gemini with up to 3 retries, accepting only responses that end with sentence-terminating punctuation.

gemini_product_summary() - app.py

```
def gemini_product_summary(flagged: list[dict]) -> str:
    if not _gemini_key() or not flagged:
        return ""
    lines = [
        "- " + r["name"] + " | Risk: " + r["risk"] +
        " | Family: " + r["fam"] + " | Concern: " + r["why"]
        for r in flagged[:20]
    ]
    prompt = (
        "You are a cosmetic safety expert explaining ingredients to an everyday consumer.\n"
        "Flagged ingredients (high to low risk):\n" + "\n".join(lines) +
        "\n\nWrite 3-5 sentences: overall risk level, 1-2 worst ingredients explained simply,
"
        "repeat families if any, one practical tip. "
        "Plain English, no bullet points, one paragraph. "
        "IMPORTANT: Always write complete sentences. Never stop mid-sentence."
    )
    for _ in range(3):  # retry up to 3 times if response looks incomplete
        result = _gemini_call(prompt, max_tokens=600)
        if result and result[-1] in ".!?":
            return result
    return result or ""



# --------------------------------
# OCR helpers
# --------------------------------

import cv2, numpy as np
```

A.8 Generative AI - Per-Ingredient Explanation with Caching

Generates a constrained two-sentence plain-English explanation per ingredient using Gemini 2.5 Flash. Applies the same retry logic and caches successful responses in Streamlit session state.

gemini_explain_ingredient() - app.py

```
def gemini_explain_ingredient(name: str, risk: str, family: str, why: str) -> str:
    cache_key = "_expl_" + re.sub(r"[^a-z0-9]", "_", name.lower())
    if st.session_state.get(cache_key):
        return st.session_state[cache_key]
    prompt = (
        f"You are a cosmetic safety expert talking to someone who knows nothing about
chemistry.\n"
        f"Ingredient: {name}\nRisk: {risk}\nFamily: {family}\nConcern: {why}\n\n"
        f"Write exactly 2 sentences:\n"
        f"1. What this ingredient IS and what it does in a beauty product\n"
        f"2. Why someone should or shouldn't be concerned - be specific\n"
        f"Plain English only, no jargon. No bullet points, just 2 sentences. "
        f"IMPORTANT: Always write complete sentences. Never stop mid-sentence."
    )
    for _ in range(3):  # retry up to 3 times if response looks incomplete
        result = _gemini_call(prompt, max_tokens=500)
        if result and result[-1] in ".!?":
            break

    if result:
        st.session_state[cache_key] = result
    return result or ""
```

A.9 Database - Hazard Record Retrieval

Retrieves all hazard records for a given ingredient ID joined with hazard classification metadata and regulatory source citations, ordered by severity descending.

hazards_for() - app.py

```
def hazards_for(ingredient_id: int):
    with closing(CON.cursor()) as cur:
        cur.execute("""
            SELECT h.code, h.category, h.description, h.severity,
                   ih.notes,
                   s.name AS source_name, s.url AS source_url
            FROM ingredient_hazards ih
            JOIN hazards h ON h.id = ih.hazard_id
            LEFT JOIN sources s ON s.id = ih.source_id
            WHERE ih.ingredient_id = ?
            ORDER BY COALESCE(h.severity,0) DESC, h.category, h.code;
        """, (ingredient_id,))
        return [dict(r) for r in cur.fetchall()]
```